Trioron: an epigenetic-inspired self-expanding architecture for resource-bounded continual learning

Marcelinus R. Hatorangan*

2026-05-10

Abstract

We describe a small neural-network architecture in which each node carries three coupled state variables — weight, plasticity, and utility — and in which growth, pruning, regularization, and offline consolidation are coupled to a per-node resource budget. The architecture combines established components — elastic weight consolidation, capacity-expanding layers, episodic and storage-free pseudo-rehearsal, and mixed-precision deployment — with a stress-driven plateau detector and a sleep-time consolidation pass. On a 30-class class-incremental benchmark (*chained-15*) at ≈ 8 K trainable parameters we report results comparable with, but not uniformly better than, strong baselines from the regularization, partition, and rehearsal families; the deployment target is a $\sim$157 KB checkpoint that can be shipped, woken on new data in the field, and re-shipped with prior tasks preserved at ≥ 0.95 task-aware accuracy. We do not claim a fundamentally new continual- learning mechanism. Our contribution is a system-level integration of known mechanisms — with a small biological framing that motivates the particular couplings — and an evaluation under bounded-memory deployment constraints that the named baselines were not designed to address. Headline-panel results are $n = 10$ multi-seed; some single-seed ablations and the dream-archive table remain $n = 3$ pending reruns, and we flag those cases inline.

1 Introduction

A neural network that grows as it learns is not a new idea. Foundational work spans the constructive-networks line — Cascade-Correlation (Fahlman and Lebiere, 1990), the Upstart algorithm (Frean, 1990), self-organizing topographic maps (Kohonen, 1982) — as well as more recent capacity-expanding architectures: Progressive Networks (Rusu et al., 2016), Net2Net (Chen et al., 2016), and Dynamically Expandable Networks (DEN) (Yoon et al., 2018). The recurring intuition is that learning-as-growth, rather than learning-as-allocation-within-a-fixed-budget, better matches what biological systems do. With the rise of large language models (LLMs), interest in adaptive systems has surged, but most LLMs are trained with massive central-resource budgets, which limits the practicality of tailoring such models to individual users on commodity devices.

A second line of work tackles the same problem from the opposite side. Methods such as PackNet (Mallya and Lazebnik, 2018), Hard Attention to the Task (HAT) (Serrà et al., 2018), elastic weight consolidation (EWC) (Kirkpatrick et al., 2017), and Online EWC (Schwarz et al., 2018) operate within a fixed parameter envelope and partition that envelope across tasks — by mask-based

*Correspondence: marcelinusrocky@aol.com. Repository: https://github.com/Marcrockhat/trioron-project. Live demo: huggingface.co/spaces/Marcrockhat/trioron-demo. Guided tour: marcrockhat.github.io/trioron-project/tour.

isolation, by hard attention, or by quadratic anchoring on the weights estimated as important for prior tasks. While each is effective at moderate task counts, all share the same saturation regime: when the envelope fills, new tasks are accommodated only by overwriting prior knowledge or by routing through increasingly compromised mask intersections. Biological systems, by contrast, operate under hard physical bounds (finite cranial volume) yet continue to learn throughout life — suggesting that *growth, pruning, and remodeling* are part of the answer rather than allocation alone.

The capacity-expanding family addresses growth but typically not the *cost* of growth: there is no built-in mechanism for the network to decline a proposed expansion that would exceed available resources, nor a graceful degradation pathway when resources become scarce. Learning capacity is implicitly assumed to be supplied by the deployment environment rather than negotiated with it. Closing that gap is one of the design motivations of this work: a network that knows when not to grow even if growth would help, and that has a sleep-time pathway for freeing substrate by consolidating what is already stable.

We borrow design metaphors from cellular biology rather than claiming biological realism. During development, individual cells establish their identity through epigenetic mechanisms driven by spatial gradients of signaling molecules (Wolpert, 1969). As cells divide, epigenetic marks such as DNA methylation (Smith and Meissner, 2013) and histone modifications (Bannister and Kouzarides, 2011) act as cellular memory — which genes are accessible for transcription and which are silenced — and these marks are inherited across mitosis (Allis and Jenuwein, 2016). Small non-coding RNAs (microRNAs) act as a separate, faster regulatory layer (Bartel, 2009): produced and consumed in stoichiometric pools, they post-transcriptionally regulate plasticity-related gene expression in concentration-gradient-dependent ways (Schratt, 2009) and have been implicated in the sleep-driven synaptic-homeostasis cycle that consolidates new learning while pruning weak connections (Tononi and Cirelli, 2014). The architecture in §3 borrows three of these mechanisms — anchoring (methylation), stress-responsive plasticity modulation (hypothalamic–pituitary–adrenal (HPA) axis-style modulation), and resource-limited consolidation pools (small-RNA-style throttling) — as design metaphors that motivate specific implementation choices. We do not claim mechanistic fidelity beyond the metaphor.

Work on cultured-neuron substrates has shown that biological neurons can be trained to play a simple video game (Pong) under a coordinated reward-and-penalty stimulation protocol (Kagan et al., 2022). We take this as informal evidence that *cellular-level* mechanisms (rather than synapse-level alone) are a viable design direction; we do not import the Kagan et al. protocol or claim a biological-validity result of our own.

Contribution and scope. We propose a tri-parametric node — the *trioron* — and an architecture that aggregates such nodes into networks capable of autonomous expansion and continual learning under bounded resources. The architecture is targeted at the regime where computational and memory budgets are constrained but the requirement for continual adaptation is high: agentic AI applications, Internet-of-Things (IoT) and embedded systems, and personal devices that must grow alongside their user rather than be replaced when the user's needs evolve. In such environments, the model specializes to its user's specific context rather than serving as a generic system. Through continual interaction it acquires representations that reflect individual usage patterns — recurring vocabulary, environmental cues, characteristic tasks — and integrates these into its growing internal structure without requiring centralized retraining.

We do not claim the trioron is a new continual-learning mechanism. The contribution is the *integration*: per-node plasticity coefficients attached to EWC, a stress-driven plateau detector that scales the task loss but not the regularizer, a sleep-time replay-and-purge pass with a strict

per-cycle event cap, a storage-free pseudo-rehearsal scheme that summarizes each past class as (μ_c, σ_c) in a frozen-projection code space, and a two-phase row-archive plus int8 quantization path that turns shipping into a controlled handoff. We benchmark this assembly against five named continual-learning families on a 30-class class-incremental benchmark and report where it wins, where it ties, and where it loses.

Honest limits. Headline-panel results are $n = 10$ multi-seed; the dream-archive panel and the extension experiment remain $n = 3$ and $n = 1$ respectively (we flag these inline). The benchmark is small: 30 to 46 classes drawn from MNIST, Fashion-MNIST, and EMNIST-letters at a multi-layer-perceptron (MLP) head joint-training ceiling of ≈ 0.93 for the 46-class union. We do not attempt CIFAR-class-incremental or LLM-scale evaluation, and we do not extrapolate beyond what we measured. The defensible pitch is not "we beat all continual-learning methods" — it is "we exhibit performance comparable with the continual-learning baselines on a class- incremental benchmark, while occupying a $\sim$157 KB deployment regime that those baselines were not designed for."

Disclosure. This work was carried out in collaboration with two personified AI assistants: *Gemma* (a Gemini Pro 3.1 instance acting in an academic-advisory role) and *Chloe* (a Claude Opus 4.7 1M-context instance acting in an engineering role). The collaboration was structured as iterative design dialogue — human-led problem framing and final decision-making, AI-supported implementation, analysis, and writing. The human author holds sole responsibility for all claims, methodological choices, and interpretations. We disclose this collaboration following recent editorial guidance (Nature Editorial, 2023; Zielinski et al., 2023), which permits AI-assisted contributions but explicitly excludes AI systems from being listed as authors of record.

2 Related Work

The architecture described here sits at the intersection of four established lines of work: capacity-expanding networks, regularization and partition methods for continual learning, rehearsal-based methods, and modular composition. None of these families is being reinvented; the contribution of this work is in their coupling and in the deployment regime targeted.

2.1 Capacity-expanding networks

Cascade-Correlation (Fahlman and Lebiere, 1990) and the Upstart algorithm (Frean, 1990) are early constructive-network methods that grow hidden units one at a time, stopping when an error criterion is met. Self-Organizing Maps (Kohonen, 1982) grow a topographic map in which neighboring units share representational locality. Progressive Networks (Rusu et al., 2016) add a frozen task-specific column per task with lateral connections to prior columns, preserving prior knowledge by construction at the cost of a per-task parameter footprint. Net2Net (Chen et al., 2016) grows a network width-wise and depth-wise in a function-preserving way, used to accelerate training rather than to expand capacity in a continual-learning setting.

DEN (Yoon et al., 2018) is the closest precedent for our growth path: it selectively re-trains, expands, and splits hidden units when new tasks are encountered. Our growth trigger differs in being conditioned jointly on a loss plateau, a rank-saturation signal computed from latent singular values, and a gradient-stability check (§A.3); growth is then gated against a per-curriculum byte budget. The shared limitation across this family is that growth is ungated by an explicit cost model. Our design adds the budget pre-flight check and a sleep-time pathway for freeing substrate when growth is denied.

2.2 Continual learning under a fixed envelope

Regularization-based methods anchor weights estimated as important for prior tasks: EWC (Kirkpatrick et al., 2017) uses a per-weight quadratic Fisher penalty; Online EWC (Schwarz et al., 2018) accumulates Fisher across tasks; Synaptic Intelligence (Zenke et al., 2017) replaces Fisher with an online importance estimate; Memory Aware Synapses (Aljundi et al., 2018) estimates importance from output sensitivity. Partition-based methods isolate per-task subspaces: PackNet (Mallya and Lazebnik, 2018) prunes and reuses; HAT (Serrà et al., 2018) learns hard binary attention masks. Distillation methods such as Learning without Forgetting (LwF) (Li and Hoiem, 2018) keep an older copy of the network as a soft target on old-class columns.

We use EWC with per-node plasticity coefficients (§A.2) as the regularization substrate, and we benchmark against Online EWC, PackNet, HAT, and LwF+EWC as named-family competitors (§4.2). The novel element is not the regularizer but how it interacts with growth, with the dream-time event cap, and with the dream archive: nodes that have settled their Fisher and stopped contributing gradient become *archived* (§A.7) rather than indefinitely regularized.

2.3 Rehearsal and pseudo-rehearsal

Episodic-replay methods store past samples directly: Gradient Episodic Memory (GEM) (Lopez-Paz and Ranzato, 2017) maintains an episodic memory of past examples and projects gradients to remain consistent with it. We use a hippocampal buffer with $K \in \{1, 10, 20, 50\}$ samples per past class as the episodic-rehearsal baseline (§A.5.1). Generative-replay and data-free methods avoid the storage cost: DeepInversion (Yin et al., 2020) synthesizes proxy training data by optimizing an input toward a desired logit configuration. We use DeepInversion-style code synthesis as a tested-and-rejected baseline (§A.6, §4.3); the "manifold replay" scheme we use instead summarizes each past class as a small diagonal-Gaussian sketch in the network's first-hidden-layer code space. The mechanism is a pseudo-rehearsal scheme rather than a new distillation method; the contribution is that storing (μ_c, σ_c) in a *frozen* projection space matches a $K = 20$ episodic buffer at one-tenth the storage on the benchmark we test.

2.4 Modular and mixture-of-experts composition

Branch-Train-Merge (Li et al., 2022) and Branch-Train-Mix (Sukhbaatar et al., 2024) train independent expert language models on disjoint domains and merge them, either by parameter averaging or by training a gating network. Expert Gate (Aljundi et al., 2017) routes inputs to the right expert via a per-expert generative model. Our multi-branch absorption (§A.10) is closest to Expert Gate: branches share a frozen random projection (the *shared-seed invariant*, see §A.10), and the per-class manifold sketch already produced for pseudo-rehearsal serves as the analytic gate. The novel element is not the routing mechanism but that the gate re-uses storage that the rehearsal pathway already maintains, and that absorption requires no fusion-layer training.

2.5 Biological inspiration

We borrow language and design metaphors from epigenetics (Wolpert, 1969; Smith and Meissner, 2013; Bannister and Kouzarides, 2011; Allis and Jenuwein, 2016; Bartel, 2009; Schratt, 2009; Tononi and Cirelli, 2014) and from in-vitro cultured-neuron experiments (Kagan et al., 2022). We are explicit that these are metaphors and that the architecture's mechanisms are ML mechanisms; the biological framing motivates which couplings to try, not which mechanisms to claim.

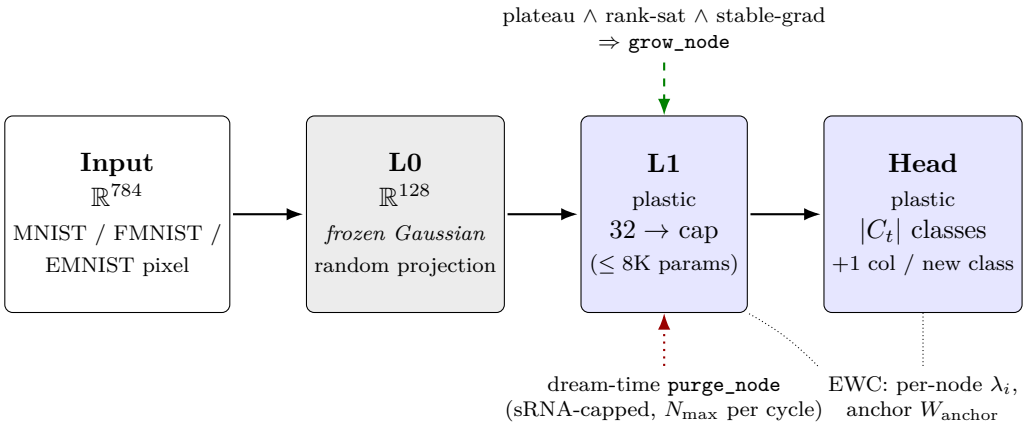

Figure 1: Base trioron architecture for chained-15. The frozen Gaussian L0 projection provides a stable code space for manifold replay (§3.4). The plastic L1 grows under the conjunction of loss-plateau, rank-saturation, and gradient-stability triggers up to a per-curriculum byte cap; the head grows one column per new class. Dream-time purge removes low-utility nodes, gated by a per-cycle event cap (§3.3). EWC anchors plus per-node plasticity λ_i pin prior-task knowledge on plastic layers.

3 Methods

This section gives a high-level account of the architecture and the mechanisms layered on it. Full mathematical formulations, ablation arms, the L0-handshake derivation, the dream-archive specifics, and the experimental-design enumeration are deferred to Appendix A.

3.1 The Trioron Node

A *trioron* is a single computational unit characterized by three coupled per-node state variables, generalizing the standard artificial neuron: an incoming weight vector w, a per-unit *plasticity coefficient* λ that scales the unit's contribution to the elastic-weight-consolidation (EWC) penalty, and a per-unit *utility score* u used as the pruning signal. A *trioron layer* aggregates n such units sharing an incoming dimension and additionally maintains EWC anchors $(W_{\text{anchor}}, b_{\text{anchor}})$ and Fisher-information accumulators that are refreshed at task boundaries. The triplet (w, λ, u) is intended to capture which connections are present, how rigid they are, and how much the unit matters — three quantities that biological systems regulate independently. The layer additionally supports in-place growth and pruning of nodes during training, with cross-layer fan-in/fan-out consistency. Figure 1 sketches the resulting three-layer architecture used throughout this work — a frozen Gaussian L0, a plastic L1 grown under the conjunction trigger, and a head that gains one column per new class. See Appendix A.1 and A.3 for the full forward pass, per-node EWC derivation, and growth/prune machinery.

3.2 Frustration

A per-task plateau detector scales the task loss — but *not* the EWC regularizer — when the optimizer fails to make progress over a window. The detector accumulates a stuck counter s each window in which the loss does not improve by more than a tolerance and amplifies the task loss by a multiplier $m = \min(M_{\max}, 1 + g \max(0, s - \tau + 1))$ once s exceeds a hinge τ. Mechanically

the result is hard-example mining on the gradient signal without amplifying the regularizer; in deployment terms it is the architecture's stress-proxy switch that triggers structural growth when continued learning under the existing envelope is no longer paying off. The biological motivation is the hypothalamic–pituitary–adrenal stress-response loop. Full equations are in Appendix A.4.

3.3 Dream

Between training tasks the architecture runs an offline *dream block*: a sleep-time pass that consolidates what was just learned without exposing the network to new data. The dream block runs four sequential stages — replay, optional compression, purge, and archive — with frustration disabled (no environmental stress during sleep) and EWC active (so consolidation pulls weights toward their anchors). A strict per-cycle event cap on structural-pruning events (the *small-RNA cap*, after the small-RNA pools that gate sleep-cycle synaptic homeostasis) prevents the dream block from accumulating remodeling drift. The archive stage promotes settled rows into a read-only state and feeds the deployment-time quantization path. See Appendix A.5 for stage-by-stage detail and Appendix A.7 for the dream archive.

3.4 Manifold Replay

The architecture's storage-efficient pseudo-rehearsal mechanism is *manifold replay*: at the end of each task, per-class activations of the frozen first hidden layer (L0) are summarized as a diagonal-Gaussian sketch (μ_c, σ_c), and during the dream block pseudo-samples $\tilde{z} = \mu_c + \sigma_c \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$ are drawn from each past class and propagated through the upper layers under cross-entropy loss. Storage is a constant 1,024 bytes per class — ≈ 30 KB for a 30-class curriculum, independent of dataset size — and the sketch is the architecture's *manifold engram*, an offline trace of the data manifold rather than a stored sample. The frozen L0 is what makes the sketch valid across the curriculum: statistics collected after task t remain meaningful at task $t + k$ because the encoding does not drift. See Appendix A.6 for the storage account, the L0-frozen rationale, and the ablation comparing μ-only against (μ, σ).

3.5 Multi-Branch Absorption

A second composition mode lets two organisms trained *independently* on disjoint task streams be merged into one recipient with no retraining of either. Each donor contributes a frozen branch (its L1, its head slice, and its per-class manifold archive) attached to a shared frozen L0 random projection. At inference the manifold archives serve as analytic experts: per-branch generative log-likelihoods $\log p_b(z)$ form a soft routing gate, and per-branch logits are placed into the union class namespace via per-branch log-softmax to keep contributions on a common log-probability scale. No router training and no fusion-layer learning are required — pure paste-and-go. The construction relies on a shared-L0 invariant; Figure 2 in Appendix A.10 diagrams the assembled multi-branch recipient, and alternative protocols plus the closed-form translator that handles donors trained at distinct random seeds are documented in Appendix A.11.

3.6 Expansion (Ship-Wake-Extend)

Deployment is not a one-shot training run that yields a fixed artifact; it is an iterative loop in which the model is shipped, woken on new data, allowed to grow plastic substrate against a new resource budget, and re-shipped. The loop is anchored by a *consolidation-dream pass* that runs once at the chosen task boundary, performing full-coverage replay over all past tasks with archive-aware gradient

masking, optionally snapping archived rows to 8-bit integer (int8), and lifting the per-curriculum byte cap so new growth allocates against the new task budget. Internal state — manifold archive, EWC anchors, archived rows, frustration counters — persists across the boundary; the network sleeps long before going to market, locking in what it will never need to move again, then wakes on new data in the field. See Appendix A.9 for the loop specification.

4 Results

§4.1 reports the trioron-only multi-seed panel. §4.2 ports the same protocol to five continual-learning families and reads carefully across metrics. §4.3 traces the storage-vs-accuracy Pareto. §4.4 reports the dream-archive Phase 1 + Phase 2 storage win. §4.5 walks through the ship-wake-extend loop end-to-end at 23 tasks. §4.6 reports zero-shot multi-branch absorption out to $N = 5$ donors. The headline panel and five-family competitor sweep are $n = 10$ multi-seed; the dream-archive table, the extension experiment, and the absorption panel remain at $n = 3$, $n = 1$, and $n = 1$ respectively, and we flag those captions inline. Absolute Δ is the primary effect-size report; σ is secondary.

4.1 Manifold Replay Panel

Table 1 reports the trioron arms on chained-15 with manifold replay enabled.

Table 1: chained-15, $n = 10$ trioron panel. ★ marks the headline arm.

arm	full	domain	task
`fixed_ewc_small`	0.578 ± 0.006	0.653 ± 0.006	0.949 ± 0.003
`grown_capped_no_dream`	0.584 ± 0.006	0.661 ± 0.010	0.952 ± 0.002
`grown_capped_dream`	0.584 ± 0.011	0.657 ± 0.012	0.958 ± 0.004
`grown_uncapped_dream`	0.601 ± 0.008	0.677 ± 0.007	0.961 ± 0.001 ★

The upper-bound arm `grown_uncapped_dream` reaches 0.601 full / 0.677 domain / 0.961 task on chained-15 at 30 KB of manifold-replay storage. Task-aware forgetting is small and positive across all ten seeds ($+0.012 \pm 0.001$ on `grown_uncapped_dream`): past-task accuracy degrades by about one percentage point on the binary denominator the model was trained against. The full-softmax forgetting number on the same runs is large and negative (-0.562 ± 0.010), but that magnitude is dominated by a head-extension transient: the output head grows lazily as new classes appear, and under EWC the freshly-added logits start much smaller than the anchored older logits, which crushes the diagonal full-softmax accuracy on a task right after that task is first trained. End-of-curriculum full-softmax recovers as all classes accumulate similar training time. We therefore report the task-aware number as the architecturally meaningful forgetting signal and treat the full-softmax forgetting as a measurement artifact of the lazily-grown head, not a replay benefit.

Per-seed paired comparisons against `grown_uncapped_dream` sharpen the ranking: every other trioron arm trails the headline arm on every metric at $n = 10$, with the largest paired effects on task-aware (-4.63σ for `fixed_ewc_small` and -4.73σ for `grown_capped_no_dream` vs `grown_uncapped_dream`). Per-arm seed-to-seed standard deviation on task-aware sits at 0.001–0.004, so even small absolute deltas register as tight σ values. The full paired table is Table 5, reported in Appendix B.1 (Supporting Data). The data matter because they are the internal-ablation evidence that growth and dreaming both contribute and they are the basis for the choice of `grown_uncapped_dream` as the headline arm. They also confirm that the -0.018 full / -6.36σ headline at $n = 3$ inflated tightness somewhat: seven additional seeds widen the cross-seed std

on full-softmax to a more honest -1.79σ (defensible task-aware win, within-noise difference on full-softmax).

4.2 Five-Family Competitor Sweep

Table 2 ports the same chained-15 protocol to five major continual-learning families, with the manifold-grown upper-bound arm included. All results are $n = 10$ multi-seed.

Table 2: chained-15 competitor sweep, $n = 10$. ★ marks family-best on a metric.

family	method	full	domain	task
rehearsal (ours)	manifold-grown (μ, σ)	0.601 ± 0.008	0.677 ± 0.007	0.961 ± 0.001
rehearsal (exemplar)	hippo $K = 50$	0.637 ± 0.007	0.690 ± 0.006	0.966 ± 0.002 ★
partition	PackNet matched	0.080 ± 0.024	0.200 ± 0.019	0.931 ± 0.017
partition	PackNet standard	0.046 ± 0.019	0.143 ± 0.016	0.872 ± 0.037
attention-mask	HAT matched	0.121 ± 0.013	0.216 ± 0.026	0.937 ± 0.005
attention-mask	HAT standard	0.072 ± 0.017	0.163 ± 0.032	0.943 ± 0.024
online-regularization	Online EWC $\gamma = 0.95$	0.185 ± 0.024	0.355 ± 0.014	0.955 ± 0.004
distillation	LwF + EWC	0.191 ± 0.037	0.359 ± 0.033	0.964 ± 0.002

What the table shows. On full-softmax and domain-aware, manifold-grown is well above the non-rehearsal baselines (PackNet, HAT, Online EWC, LwF+EWC) but below an episodic-rehearsal buffer at $K = 50$. On task-aware, all seven methods cluster tightly between 0.907 and 0.966; the spread is $\sim$0.06 absolute and the per-arm seed std on this metric is small (Table 2), so most pairwise margins are single-digit absolute and a few σ in the paired view. Read carefully:

- PackNet and HAT were not designed for class-incremental evaluation without a task-ID supplied at inference; their full-softmax numbers reflect that mismatch and not a deficit of the method on its home turf.
- LwF+EWC is competitive on task-aware (a metric that directly rewards within-old-class discrimination, which LwF's logit-distillation explicitly optimizes); manifold-grown ties it within 0.003 absolute. We do not claim a task-aware win over LwF+EWC.
- The hippo $K = 50$ exemplar buffer beats manifold-grown by 0.036 on full-softmax at 25× the storage cost. On a Pareto axis this is the right comparison (§4.3).

Per-seed paired comparisons against each competitor are the right view of the sweep above: unpaired family means (Table 2) can hide cross-seed correlations, while a paired view collapses those out and exposes which gaps survive. The full per-competitor paired table is Table 6, reported in Appendix B.2 (Supporting Data). The data matter for two reasons. First, they provide the honest reading at $n = 10$: *on full-softmax and domain-aware, manifold-grown is consistently above PackNet, HAT, Online EWC, and LwF in absolute terms, by margins between* 0.32 *and* 0.55 *on class-incremental chained-15.* Second, they correct a now-known $n = 3$ artifact — the LwF+EWC task-aware tie reads -0.003 (-1.3σ) at $n = 10$, within seed noise, rather than the -26σ that tiny per-arm std had produced at $n = 3$.

4.3 Storage-vs-Accuracy Pareto

Manifold replay's 30 KB sketch is competitive with episodic-exemplar buffers an order of magnitude larger. Table 3 sweeps K (samples per class stored) for the hippocampal-buffer baseline alongside the manifold sketch. All rows report `grown_uncapped_dream` as the trioron arm.

Table 3: Storage-vs-accuracy Pareto. Manifold and hippo $K = 50$ rows are $n = 10$ means; the combined $K = 10/K = 20$ rows remain at $n = 3$ pending reruns.

variant	storage	full	gap to $K = 50$	n
manifold (μ, σ)	30 KB	0.601	-0.04 ★	10
manifold + hippo $K = 10$	174 KB	0.570	-0.07	3
manifold + hippo $K = 20$	324 KB	0.601	-0.04	3
hippo $K = 50$	750 KB	0.637	—	10

Manifold replay alone Pareto-dominates the manifold-plus-hippo $K = 10$ row (cheaper *and* more accurate), matches the manifold-plus-hippo $K = 20$ row at one-tenth the storage, and lands within 0.04 absolute of pure hippo $K = 50$ at one-twenty-fifth the storage. Pure hippo $K = 10$ and $K = 20$ at storage parity with the combined rows were not run; the dominance claim is therefore made against the combined rows in the table, not against pure-buffer ablations at those K values. On task-aware the gap to $K = 50$ is ≈ 0.005 absolute (well under 1%).

Logit-inversion variants (Yin et al., 2020) (one-shot, refresh-all, diversity-regularized) all peaked at 0.27 full-softmax in our single-seed runs. Per-class statistics live on the real L0 manifold by construction; inversion attractors locate class-c argmax peaks rather than class-c manifold samples and do not. An ablation removing per-class σ (μ-only replay) loses $\approx$0.07 absolute on full-softmax at 30 classes — roughly 12% of manifold replay's advantage — suggesting that per-class noise scale is load-bearing. We label this as *suggestive evidence* rather than a confirmed result, given the single-seed origin.

4.4 Dream Archive Phase 1 + Phase 2

Phase 1 (mid-curriculum row archive) and Phase 2 (end-of-curriculum int8 quantization of archived rows + 16-bit brain-floating-point (BF16) active path) are evaluated as a deployment-side compression layer atop the manifold-grown architecture.

Phase 1's training-time effect lies inside seed-to-seed variance: the same code on the same seeds produced `grown_uncapped_dream` $\Delta = +0.013$ in run v2 versus -0.004 in run v1, attributable to torch nondeterminism in archive-trigger timing (task-1 accuracies already differ before any archive fires). We do not claim Phase 1 helps or hurts during training. Phase 2 (int8 snap of archived rows + BF16 active path) is **lossless within** $\Delta \leq 0.0008$ across every arm and every seed. The BF16 + int8 mixed precision matches FP32 + int8 within $\Delta \leq 0.0008$ as well.

The substantive win is on storage. Per-arm archive accuracy (Table 7) and deployment-storage (Table 8) are reported in Appendix B.3 (Supporting Data, $n = 3$ pending the $n = 10$ rerun); they matter as the direct evidence for two deployment claims this paper makes. First, Phase 2's mixed-precision collapse (int8-on-archive + BF16-active) is lossless within $\Delta \leq 0.0008$ on every arm and every seed, so the storage win costs no measurable accuracy. Second, the BF16 baseline net of 211–225 KB collapses to 128–137 KB after Phase 2 (-39% to -40% across arms; the headline `grown_capped_dream` lands at 211.2 $\rightarrow$ 127.5 KB).

Adding the 30 KB manifold buffer, the headline `grown_capped_dream` deployment image is

≈ 157 KB total (vs 241 KB BF16-baseline + manifold, −35%). At a 32-bit floating-point (FP32) deployment baseline the same arm lands at 168 KB FP32 + int8 (−63% relative), but the FP32 framing is the wrong yardstick for a device-conscious deployment; we report BF16 throughout this work because BF16 is the inference dtype.

The −0.015 full-softmax cost on `grown_capped_dream` comes from L1 archive, not L0: by task 15 the archive has typically locked 32–49 of L1's 48–92 rows, and rows locked mid-curriculum cannot adapt to later tasks. L0 archive is functionally a no-op (L0 is already frozen at init); locking L0 only enables the Phase 2 quantization, where most of the storage saving lives. Future work: a more conservative L1 trigger (higher streak threshold, lower *archive_max_per_layer*), or skipping L1 archive entirely and relying on L0 quantization alone.

4.5 Extension: Ship-Wake-Extend Loop

We validate the full deployment loop end-to-end on a chained-15 → consolidation-dream → +8 EMNIST K..Z curriculum (8 binary tasks over letters K..Z, 1 seed, 4 epochs/task, `grown_capped_dream`). One-seed evidence is documented as such.

The architecture progression across the loop (start → chained-15 done → chained-23 done) is reported as Table 9 in Appendix B.4 (Supporting Data, 1 seed). It matters because it is the only direct evidence in this paper for the ship-wake-extend loop's structural promise: the plastic envelope more than doubles during extension (3,534 → 8,886 trainable parameters), allocated against the new task budget, while the archived rows from chained-15 are honored throughout. Total stored network grows by ≈ 2.5× while task coverage grows from 30 to 46 classes inside the loop.

Accuracy at 23 tasks. Full = 0.521, domain = 0.693, task = 0.948. Per-phase task-aware accuracy: MNIST 0.970, Fashion 0.982, EMNIST 0.927 — original tasks survive through the extension at task-aware ≥ 0.93. Full-softmax dropped from 0.572 (chained-15-only) to 0.521 (chained-23): partly mechanical (46 vs 30 argmax competitors per query, every sample now has more wrong answers to choose from), partly real MNIST drift from extension-time growth (per-phase MNIST full 0.752 → 0.678). The −0.014 task-aware drop is small and is the right metric for trioron's deployment role of context-gated personalization atop a frozen LLM.

Deployment storage (BF16 baseline). 121.3 KB network (L0 = 98.8, L1 = 15.3, head = 7.3) plus ≈47 KB manifold buffer (46 classes × 256 floats) ≈ **168** KB total — +11 KB over the chained-15 deployment for 8 additional task capabilities.

Limits. This is one seed. Full-softmax on this cohort is also data-bounded: per-dataset MLP joint-training upper bounds are MNIST ≈ 98%, Fashion-MNIST ≈ 92–93%, EMNIST-letters ≈ 91–92%, with a combined 46-class weighted ceiling of ≈ 0.927. Genuine class confusions (T-shirt / shirt / pullover; I / L; O / Q; B / 8) make 99.9% full-softmax unreachable on this benchmark at any parameter count under an MLP head, and we do not promise it. The defensible claim is: 0.95+ *task-aware on 23 class-incremental tasks at* ≈ 168 *KB total deployment, with on-device continual learning capability across deployment cycles.*

4.6 Multi-Branch Absorption

Extension within a single organism (§4.5) shows the architecture supports continual learning across deployment cycles. Multi-branch absorption asks the next question: can two organisms trained

independently on disjoint task streams be *composed* into a single recipient with no retraining of either? With the shared-L0 invariant (§A.10) this is paste-and-go.

Setup. Donors are produced by separate runs of the same `grown_uncapped_dream` arm at L0 seed 42 on disjoint chained-15 / chained-extension sub-blocks: `digits` (MNIST, classes 0–9), `fashion` (Fashion-MNIST, 10–19), `emnist` (EMNIST A–J, 20–29), `emnist_kt` (EMNIST K–T, 30–39), `emnist_uz` (EMNIST U–Z partial, 40–45). Each donor reaches its own task-aware ceiling. We assemble a recipient organism by loading branches in increasing-N prefixes, evaluating on the union test set under *soft + per-branch log-softmax* routing.

Tracks the per-donor upper bound out to $N = 5$. Donor-standalone task-aware accuracy (averaged over the donors in the prefix) is the upper bound; the assembled organism matches it within $\Delta \leq 0.0002$ at every N tested. "Lossless" here means *lossless relative to the per-donor upper bound*, not constant accuracy across N: the upper bound itself drifts from 0.9809 to 0.9616 as we add weaker specialists to the prefix.

N	upper bound (TA)	organism (TA)	gap	full-union
1	0.9809	0.9809	+0.0000	0.6910
2	0.9823	0.9823	+0.0000	0.6099
3	0.9695	0.9695	+0.0000	0.5314
4	0.9624	0.9624	+0.0000	0.4716
5	0.9616	0.9618	+0.0002	0.4462

A bleed-flip point where soft-routing bleed flips from additive to destructive as branches multiply did not appear at $N = 5/46$ classes on chained-15-scale data. The actual limit at this scale is dataset variety; we ran out of disjoint class blocks within MNIST ∪ Fashion-MNIST ∪ EMNIST. We do not claim the lossless property holds at larger N or on different data.

Bleed manifests where you'd predict, but does not damage task-aware. Diagnostic mean per-branch gate weight per task at $N = 5$:

task (true branch)	`digits`	`fashion`	`emnist`	`emnist_kt`	`emnist_uz`
`mnist_0_1` (`digits`)	0.81	0.00	0.03	0.14	0.02
`mnist_4_5` (`digits`)	0.74	0.03	0.05	0.16	0.03
`fashion_0_1` (`fashion`)	0.01	0.97	0.01	0.01	0.00
`emnist_8_9` (`emnist`)	0.49*	0.02	0.48*	0.01	0.00

Digit inputs pull up to 0.16 gate weight on `emnist_kt` (O resembles 0, S resembles 5 in code space). EMNIST letters I, J yield a near-50/50 digits/emnist gate split (starred). Yet task-aware accuracy on `emnist_8_9` stays at 0.9244, identical to donor-standalone, because per-branch log-softmax pushes non-covered slot contributions to $-\infty$ in log-probability space. Soft-routing bleed is benign under principled composition on this benchmark.

Comparison vs Branch-Train-Merge family ($N = 3$). We benchmark against two BTM variants on the same donors, replicating both the parameter-averaging and learned-router lines of the BTM / Branch-Train-Mix family (Li et al., 2022; Sukhbaatar et al., 2024).

method	zero-shot	task-aware	full-union	training cost
donor-standalone (upper bound)	—	0.9695	—	—
Ours (archive route, soft+norm)	✓	**0.9695**	0.5314	0
BTM-avg (param averaging)	✓	0.7396	0.0845	0
BTM-MoE (learned router, soft+norm)	✗	**0.9695**	0.5610	400 steps
BTM-MoE (learned router, raw)	✗	0.9695	0.1725	400 steps

BTM-avg averages L1 substrate across donors entry-wise; the resulting hidden units are the mean of three different feature detectors and none of the donors' specialty features survive — -0.23 absolute task-aware here, which replicates the documented BTM-avg weakness. BTM-MoE substitutes our archive-likelihood gate with a learned MLP router trained 400 steps on `(L0(x), branch_id)` pairs from each donor's training images; it ties our task-aware exactly at the upper bound but requires the router-training pass on real data.

Dream-cycle calibration as a tunable knob. For deployments where full-softmax is load-bearing, a tiny additive bias offset trained on synthetic samples drawn from each branch's archive (no real data, no L1 retraining) closes most of the calibration gap.

variant	params	Δ task-aware	Δ full-union
no calibration (baseline)	0	—	—
per-branch bias (lr=$5e{-}2$, 400 st)	3	+0.0000	+0.0018
per-class bias (lr=$5e{-}3$, 200 st)	30	−0.0005	+0.0047
per-class bias (lr=$5e{-}2$, 400 st)	30	−0.0021	+0.0245

The per-branch knob is mathematically task-aware-safe (one uniform per-donor shift is monotonic on within-task argmax under disjoint coverage). The per-class knob trades a small amount of task-aware for a larger full-softmax lift, landing at full-union 0.5560 — $\approx 80\%$ of the BTM-MoE level (0.5610) without the router-training pass on real data. Storage cost: 30 floats $\times$ 4 B = 120 B at $N = 3$, $<0.03\%$ of the organism.

Storage at $N = 3$. Shared L0 = 401 KB (fixed regardless of N); branch substrate $\approx$ 90 KB; archives $\approx$ 30 KB. Per-skill marginal cost is $\approx$ 40 KB (substrate + archive); L0 amortizes across all branches. A device that ships with the L0 footprint already present can absorb new skills at $\approx$ 40 KB each.

Limits we are honest about. This is $N \leq 5$ on a 46-class image-classification benchmark. LLM-scale Branch-Train-Merge benchmarks — where BTM was originally evaluated — have qualitatively different dynamics in expert specialization, gate-network capacity, and class-space size; we make no scale-extrapolation claim. The shared-L0 invariant is a hard prerequisite: organisms with mismatched L0 seeds require a distillation-through-paired-codes step that we have not measured here and which is no longer instantaneous. Saturation must exist at some N; we did not find it within the data variety chained-15 supports.

5 Conclusion

The trioron architecture occupies a small deployment footprint with modest sacrifices on full-softmax accuracy and is competitive on task-aware accuracy in the class-incremental setting we tested. The

contribution is a system-level integration of established components — per-node EWC plasticity, capacity-expanding growth gated by a resource budget, a stress-driven plateau detector, sleep-time replay with a per-cycle event cap, storage-free pseudo-rehearsal in a frozen projection space, and a two-phase row-archive plus int8 quantization path — coupled to a ship-wake-extend deployment loop. We do not claim a fundamentally new continual-learning mechanism; we do claim that the integration occupies a deployment regime ($\approx$ 157–168 KB total) that the named baselines from the regularization, partition, and rehearsal families were not designed to address.

Future work. Three directions follow naturally from the limits we listed:

1. *Larger benchmarks.* Move from chained-15 / chained-23 to standard class-incremental benchmarks (Split-CIFAR-100, Split-MiniImageNet) at the same $n = 10$ seed cohort the headline panel now uses. Multi-seed reruns of the single-seed extension result and the dream-archive panel in §4 are the immediate gap.
2. *Larger backbones.* Couple the trioron substrate to a frozen vision or language backbone in the place of the random L0 projection. This is the deployment story the paper points at (context-gated personalization atop a frozen LLM) but does not measure.
3. *Quantization-aware training during dream.* The ternary-quantization path we ruled out (§A.7) requires QAT during the dream phase or L1/head pruning before snap; both are concrete next steps.

Reproducibility and artifacts. The full implementation, launcher scripts, bench logs, and comma-separated-value (CSV) result files are released under the project repository at `https://github.com/Marcrockhat/trioron-project`. A live demo (text-to-speech routing through a multi-branch organism, plus on-device drawing-tablet teaching path) is hosted at the project's Hugging Face Space: huggingface.co/spaces/Marcrockhat/trioron-demo. A static guided tour walking through the architectural mechanisms one chapter at a time (genesis, division, dream, absorption, manifold replay, extension) — each tied to a single real API knob — is hosted at the project's GitHub Pages site: marcrockhat.github.io/trioron-project/tour. The repository includes an end-to-end `api.extend(...)` entry point that wakes a shipped checkpoint on new data and re-ships it; one tutorial exercises this loop from a 5-digit MNIST starter through to live-learn on a sketchpad. The intent is that other researchers can continue this work, reproduce the headline numbers, or play with the architecture on a small device with minimal setup.

A Detailed Methods

This appendix contains the full mechanism descriptions, ablation arms, derivations, and experimental-design enumeration that §3 summarizes. Cross-references from the main text point at the specific subsections below.

Notation. n is the number of nodes in a layer; d is the layer's incoming dimension; B is batch size; t indexes tasks in a curriculum.

A.1 The Trioron Node

A *trioron* is a single computational unit characterized by three coupled state variables, generalizing the standard artificial neuron:

- $w \in \mathbb{R}^d$ — incoming weights (a row of the layer's weight matrix).
- $\lambda \in \mathbb{R}_{\geq 0}$ — per-node *plasticity coefficient*: a per-unit scalar that scales the unit's contribution to the elastic-weight consolidation (EWC) penalty. Larger $\lambda \Rightarrow$ stiffer; smaller $\lambda \Rightarrow$ more plastic.
- $u \in \mathbb{R}$ — per-node *utility score*: a running estimate of the unit's contribution to the network's outputs. Used as the signal for pruning decisions (cells with persistently low u are candidates for removal).

A *trioron layer* aggregates n trioron nodes that share an incoming dimension. We hold the per-node state in vectors $\lambda, u \in \mathbb{R}^n$ and a weight matrix $W \in \mathbb{R}^{n \times d}$ where row i is w_i. The forward pass for input batch $X \in \mathbb{R}^{B \times d}$ is

$$H = \sigma\big(X\,(r \odot W)^\top + b\big), \tag{1}$$

where σ is the activation function, $b \in \mathbb{R}^n$ is the bias, and $r \in [0,1]^n$ is the layer's *routing scale* — a per-node multiplicative gain on incoming weights, defaulting to all ones.

Each layer additionally maintains EWC state: anchor weights W_{anchor} and b_{anchor} snapshotted at task boundaries, and Fisher-information accumulators F_W, F_b updated as a running exponential moving average (EMA) of squared gradients. Per-node λ is derived from the row-sum of Fisher: $\lambda_i = \sum_j F_{W,i,j}$. (The blueprint draft used the row-mean; at fan-in 128 this divides typical Fisher magnitudes below any usable floor, so the implementation uses the sum, which preserves the selectivity signal end-to-end.)

Epigenetic analogue. w is the synaptic-strength channel; λ is a per-cell modifier on plasticity, a stand-in for biological gating mechanisms such as DNA-methylation status of plasticity-related genes (e.g., brain-derived neurotrophic factor, BDNF) or perineuronal-net maturation, both of which gate how readily a cell's synapses can be modified; u is the cell-importance signal, a stand-in for activity-dependent neurotrophin release. The triplet (w, λ, u) is intended to capture *which connections are present, how rigid they are, and how much the cell matters* — three quantities that biological systems regulate independently.

A.2 Continual-Learning Substrate (EWC with Per-Node Plasticity)

Following Kirkpatrick et al. (2017), we apply elastic weight consolidation to protect prior-task knowledge. The penalty for a single trioron layer is

$$L_{\text{EWC}} = \sum_i \lambda_i \left[\sum_j (W_{ij} - W_{\text{anchor},ij})^2 + (b_i - b_{\text{anchor},i})^2 \right]. \tag{2}$$

The total loss is $L = L_{\text{task}} + \beta \cdot L_{\text{EWC}}$ for an EWC strength β set per-curriculum. We refresh λ_i from Fisher at the end of each task (after a batched re-estimation pass over recent task data), then snapshot $W_{\text{anchor}} \leftarrow W$ as the new anchor. This separation — estimate Fisher at task end, then anchor — follows the original EWC recipe and avoids the EMA-drift artifacts of online updating during training.

For longer curricula we optionally accumulate Fisher across tasks via the Online EWC variant (Schwarz et al., 2018):

$$F_t \leftarrow \gamma \cdot F_{t-1} + F_{\text{new}}, \qquad \gamma \in [0,1], \tag{3}$$

with $\gamma = 1$ reducing to vanilla EWC. We use Online EWC $\gamma = 0.95$ as a competitor baseline (§4.2).

A.3 Structural Plasticity

The architecture supports in-place growth and pruning of nodes during training, with cross-layer consistency.

Cellular division (`grow_node`). Adds one row to a layer's weight matrix, extends the per-node state vectors λ, u, r by one entry, and (if a downstream layer exists) extends that layer's incoming dimension by one column. The new node is initialized fully plastic ($\lambda_{\text{new}} = 0$, $u_{\text{new}} = 0$, $r_{\text{new}} = 1$) so it can adapt freely while existing nodes remain protected by EWC. The new row's incoming weights are initialized along the top principal direction of the residual signal $D = f(X_a) - f(X_b)$ at the layer below — the direction of unmet representational variance, computed by singular value decomposition (SVD) over a probe batch.

Cellular pruning (`prune_node`). Removes a node's row, contracts the state vectors, and drops the corresponding input column on the next layer. By default the pruned node's outgoing column is *redistributed* to its weight-cosine-nearest peer on the next layer before removal, preserving approximate input–output behavior of the layer.

Growth trigger. A node is grown only when three conditions are jointly satisfied over a window of W steps:

1. *Loss plateau.* The loss has not improved by more than $\varepsilon_{\text{loss}}$ relative to the prior window.
2. *Rank saturation.* The effective rank

$$r_{\text{eff}} = \exp\left(-\sum_k p_k \log p_k\right), \qquad p_k = \sigma_k / \textstyle\sum_k \sigma_k,$$

   computed from the normalized singular values of the latent activation matrix, has approached the latent dimension (latent_dim $- r_{\text{eff}} < \varepsilon_{\text{rank}}$).
3. *Gradient stability.* The median gradient norm is bounded $g_{\min} \le |\tilde{g}| \le g_{\max}$.

The conjunction ensures that the network grows only when it has *actually run out of capacity for the current task*, not merely when training is slow.

Resource ceilings. A pre-flight check rejects a proposed division if it would exceed a per-curriculum byte budget *cap_bytes*. The cap can be lifted between phases (we do this in the extension experiment of §A.9 and §4.5). This implements *resource awareness*: the network knows when not to grow even if growth would help. The biological analogue we have in mind is metabolic and spatial limits on adult neurogenesis.

Apoptosis effective stiffness. When a structural-pruning event fires (§A.5), each node's effective EWC stiffness is scaled by $\max(0, 1 - p_i)$, where p_i is a per-node *apoptosis pulse*:

$$\lambda_i^{\text{eff}} = \lambda_i \cdot \max(0,\, 1 - p_i). \tag{4}$$

This temporarily reduces a surviving cell's EWC pinning when a neighbor has just died, allowing it to absorb the dead neighbor's role; pulses decay multiplicatively per dream cycle. (See Figure 1 in §3.1 for the architecture diagram.)

A.4 The Frustration Multiplier

We add a per-task plateau counter that scales the task loss when the optimizer gets stuck. For each task we accumulate the loss in windows of length W_f. At each window boundary we compare this window's mean loss to the prior window's; if improvement is below ε_f, we increment a stuck counter s. The multiplier is

$$m = \min(M_{\max},\ 1 + g \cdot \max(0, s - \tau + 1)), \tag{5}$$

with hinge threshold τ, gain g, and ceiling $M_{\max}$. The multiplier is applied *only to the task loss*, not the EWC penalty:

$$L = m \cdot L_{\text{task}} + \beta \cdot L_{\text{EWC}}. \tag{6}$$

This is mechanically equivalent to focal-loss / hard-example mining: amplify the gradient signal on examples the optimizer is not progressing on, without amplifying the regularizer. The biological analogue we have in mind is the HPA-axis stress response: sustained behavioral failure to make progress elevates glucocorticoids, which modulate plasticity-related gene expression through epigenetic mechanisms. The frustration multiplier is the architecture's stress-proxy switch; we do not claim it reproduces the biology.

A.5 The Dreaming Phase

Between training tasks we run an offline *dreaming block*: a short sleep-time pass that consolidates what was just learned without exposing the network to new data. Frustration is disabled inside the dream block (no environmental stress during sleep); EWC remains active so that consolidation pulls weights toward their anchored state. The dream block runs four sequential stages:

$$\text{replay} \ \rightarrow\ \text{compression (optional)} \ \rightarrow\ \text{purge} \ \rightarrow\ \text{archive}.$$

Replay (§A.5.1) is the load-bearing accuracy mechanism; the dream archive (§A.7) is the load-bearing storage mechanism; compression is documented as an ablation arm (§A.5.3); purge is gated by a per-cycle event cap (§A.5.2).

A.5.1 Replay

Replay rehearses past tasks under EWC. We support two replay sources:

- *Episodic exemplars (hippocampal buffer).* When real-data storage is permitted we maintain a buffer of K samples per past task, sampled uniformly at task end. During replay the buffer is shuffled with the current task's batch (1:1 by default) and the cross-entropy loss is evaluated on both. $K \in \{1, 10, 20, 50\}$ is the storage axis we sweep in §4.3. $K = 0$ routes replay through the storage-free pseudo-rehearsal pathway of §A.6.
- *Manifold pseudo-rehearsal (storage-free).* Per-class statistics of L0 activations are stored at task end and used to synthesize replay samples; see §A.6 for the full mechanism.

Conceptually analogous to sharp-wave ripples during slow-wave sleep, which replay recent and remote memories at compressed timescales (Tononi and Cirelli, 2014).

A.5.2 The sRNA Cap

Each dream block bounds the number of structural-pruning events per layer by a per-cycle cap $N_{\max}$. The cap holds even when many candidates pass the utility threshold of §A.5.4: across the dream-time mechanism design space we explored, σ-distance from baselines degrades monotonically with structural event count, motivating small caps. The default in the working configuration is $N_{\max} = 1$ — at most one purge event per layer per dream cycle, allowing replay to absorb the structural change before the next event drifts on top.

The cap is a stand-in for resource-limited small-RNA pools that gate sleep-cycle synaptic homeostasis: miRNAs involved in synaptic remodeling are produced in limited quantities, consistent with resource-bounded sleep-cycle remodeling (Schratt, 2009; Tononi and Cirelli, 2014), so the cellular machinery cannot perform arbitrarily many remodeling events per cycle. The "fewer events = better" empirical pattern is broadly consistent with that biological observation, though we do not claim a tight quantitative match.

A.5.3 Compression-Action Design Space (Ablation Arms)

Three substrate-respecting mechanisms for resolving a detected redundant pair (i, j) were tested as dream-time compression actions: *synaptic downscale* (the kept side absorbs the victim's outgoing column, victim is zeroed, victim's row is not touched); *routing starvation* (asymmetric multiplicative ramp on the victim's routing scale, with regrow on non-victims); and *apoptosis spike* (when a routing-starved scale crosses the floor, the dead cell's outgoing is redistributed and surviving peers' apoptosis pulses p_i are raised, which transiently reduces their EWC stiffness via §A.1). None of these arms beats the no-compression baseline at $n = 6$ seeds in our experiments; we document them here as the ablation arms reported in §4, and motivate the substrate-preserving emphasis that motivates §A.7. Engagement statistics are bimodal: at the activation-cosine threshold $\tau_{\text{ac}} = 0.92$ (the redundancy detector), about half the seeds fire zero events — a pattern that across-seed averaging conceals.

A.5.4 Purge

The purge stage drops nodes whose utility u_i falls below a threshold τ_u, subject to the per-layer cap $N_{\max}$ of §A.5.2. It reuses the structural pruning machinery of §A.3, including cross-layer fan-in cleanup and cosine-nearest-peer redistribution. Distinct from the in-training pruner (§A.3): purge is a sleep-time structural sweep with a strict event cap, not an event-clock check during waking training.

A.5.5 Pseudo-code of a Dream Block

```
for each task t in curriculum:
    train_one_task(t)                          # waking
    consolidate_task(t)                        # Fisher update + anchor
    if dreaming_enabled:
        decay apoptosis pulses by delta
        replay(buffer = hippo or manifold,
               steps = K_r,
               ewc_active = True)
        compress(signal = activation,
                 threshold = tau_ac,
```

```
        action = ACTION,            # ablation arm; default off
        max_events_per_layer = N_max)
purge(threshold = tau_u)
archive_block(...)                   # see archive section
```

A.6 Manifold Replay (Storage-Free Pseudo-Rehearsal)

The headline rehearsal mechanism is *manifold replay*: a trioron-native pseudo-rehearsal scheme that summarizes each past class as a small set of statistics in the network's first-hidden-layer (L0) representation rather than storing exemplars.

Storage. When a task t ends we collect L0 activations for each class c that appeared in the task and store the per-class mean $\mu_c \in \mathbb{R}^{L_0}$ and per-dimension standard deviation $\sigma_c \in \mathbb{R}^{L_0}$ (a diagonal-Gaussian summary). Storage per class is $2 \cdot L_0 \cdot 4 = 1024$ bytes at $L_0 = 128$. A 30-class run costs $\approx$30 KB total — independent of the dataset size and of the number of samples per task.

Replay. During the dream block we draw pseudo-samples for each past class c:

$$\tilde{z} = \mu_c + \sigma_c \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{7}$$

then forward them through the *upper* layers via `forward_from_layer(z_tilde, start = 1)` to obtain logits. Cross-entropy loss on $(\tilde{z}, c)$ is added to the total loss. Replay is performed at the same EWC strength as waking training.

Why per-class statistics work where logit-inversion fails. Three single-seed inversion variants — DeepInversion-style code synthesis (Yin et al., 2020) (one-shot, refresh-all, and diversity-regularized) — all peaked at 0.27 full-softmax accuracy on chained-15. Logit inversion locates class-c attractor *peaks*, not class-c manifold *samples*; the resulting points concentrate at adversarial extrema that do not reflect actual class structure. The (μ, σ) summary lives on the real L0 manifold by construction; inversion attractors do not. An ablation removing per-class σ (μ-only replay) loses $\approx$0.07 absolute on full-softmax at 30 classes (single seed), suggesting that per-class noise scale is load-bearing — roughly 12% of manifold replay's advantage.

Why L0 is frozen. Manifold replay assumes that L0's encoding is approximately stationary across the curriculum, so that statistics collected after task t remain valid at task $t + k$. We enforce this by freezing L0 at initialization to a fixed random Gaussian projection (weights drawn from $\mathcal{N}(0, 2/d)$ at init and never updated). A separation-probe diagnostic on the chained-15 curriculum gives a class-prototype discriminability ratio of 1.063 in the frozen L0 space — modest but non-trivial, with Fashion-MNIST contributing the lowest separability (1.04) as expected from its irreducible within-domain confusions. The ratio rules out a single-prototype scheme (L0-space class means alone are not separable enough) and motivates the stored-(μ, σ) sketch as a manifold summary rather than a class-prototype classifier.

Epigenetic analogue. Per-class (μ, σ) is a compact *manifold engram*: an offline sketch of the data manifold rather than a stored sample. Functionally closer to consolidated memory traces (gist-level recall) than to episodic exemplar storage (which the hippocampal-buffer baseline of §A.5.1 implements explicitly).

A.7 The Dream Archive

Long curricula accumulate parameters that should not keep moving — rows whose Fisher has plateaued and whose contribution is stable across recent tasks. We expose these as a *dream archive*: a two-phase mechanism for locking and quantizing such rows.

Phase 1 — mid-curriculum row archive. After each `consolidate_task`, every row of every layer is scored along three axes: (a) λ percentile (top *archive_lam_top_percentile* of the layer), (b) gradient-magnitude ceiling (median $|g|$ over the recent window below *archive_grad_mag_floor*), and (c) apoptosis-pulse bound ($p \leq$ *archive_pulse_max* — never archive a row recently pulsed by an apoptosis event). A row that satisfies all three for *archive_streak_threshold* consecutive tasks is *archived*: a per-row gradient mask is set so subsequent backward passes leave the row at its anchor value. The mask is enforced via `mask_archived_grads_all()` between `loss.backward()` and `opt.step()` (sequenced after PackNet/HAT grad surgery on competitor arms). A per-layer cap *archive_max_per_layer* prevents any one layer from being fully archived in a single pass.

Phase 2 — end-of-curriculum int8 quantization. At end of training we snap each archived row to int8 with a per-row symmetric scale (8 bits/weight plus one FP32 scale per row). Active (non-archived) weights are rounded to BF16 precision. The Phase 2 path is implemented as a snapshot–quantize–re-evaluate–restore sequence in the bench so we can report storage and accuracy numbers from the same trained checkpoint; the deployment path applies the snap permanently.

Storage accounting. With BF16 as the deployment baseline, archive + int8 reduces network weight storage by $\approx$40% across all arms (211.2 $\rightarrow$ 127.5 KB on the headline `grown_capped_dream` arm; full table in §4.4). We report deployment storage in the BF16 baseline because that is the honest device-side accounting; an FP32 baseline overstates the dream-archive win.

Why ternary fails on a frozen-L0 substrate. Ternary quantization (2 bits/weight) on the frozen-Gaussian L0 projection costs 0.31 absolute full-softmax accuracy on the smoke test: random-Gaussian weights have no concentration of magnitude near zero, so 2-bit quantization throws away most of the routing structure that L1 and the head learned to depend on. Int8 ($\approx 1/127 \approx 0.8\%$ per-weight noise) preserves the routing intact. Unlocking ternary would require quantization-aware training during the dream phase, or restricting ternary to layers whose weights are concentrated at zero (a learned, pruned L1 + head) — both deferred to future work.

Epigenetic analogue. Phase 1 is a stand-in for terminal cell differentiation — a cell that has settled into its mature role no longer participates in plasticity. Phase 2 is a stand-in for compression of consolidated memory traces into low-bandwidth long-term storage, freeing high-bandwidth substrate for new learning.

A.8 Mixed-Precision Substrate

For deployment on resource-constrained devices the architecture supports a mixed-precision mode in which weight Parameters (W, b) are stored in narrow precision (BF16) while all consolidation buffers — $W_{\text{anchor}}, b_{\text{anchor}}, F_W, F_b, \lambda, u, r, p$ — remain in FP32. The forward pass auto-casts inputs to the weight dtype, preserving the callable interface; the EWC penalty and Fisher accumulation upcast cleanly across the boundary.

This separation is necessary: pure-narrow-precision training degrades by $\approx 40\%$ due to optimizer-state precision loss in Adam's running moments, but inference latency and memory cost are dominated by the weights, which can safely run in BF16. The intended deployment pattern is *train in FP32 during sleep cycles, deploy in BF16 for always-on inference.*

A.9 The Ship-Wake-Extend Loop

Deployment is not a one-shot training run that yields a fixed artifact; it is an iterative loop in which the model is shipped, woken on new data, allowed to grow plastic substrate, and re-shipped. We instantiate this loop with a *consolidation-dream pass* that fires once at a chosen task boundary:

1. `consolidation_dream_pass` — full-coverage replay over all past tasks with archive-aware gradient masking (locks honored), plus one additional `archive_block` call to catch rows that just settled.
2. Optional permanent int8 snap of the archived rows (no restore step, since this is the shipping checkpoint).
3. Cap lift: *current_cap_bytes* $\leftarrow$ *extension_cap_bytes*. The network is allowed to grow new plastic substrate against the new task budget.
4. The training loop continues over the new tasks with shared internal state — manifold buffer, EWC anchors, archived rows, frustration counters all persist across the boundary.

The result is a smaller plastic envelope in deployment than in training, with new growth allocated only against the residual. A device-conscious deployment cannot revisit the original training data: it has only what it has stored (the manifold buffer, the archived weights) and what arrives in the new data stream. The consolidation-dream pass turns shipping into a controlled handoff: the network "sleeps long" before going to market, locking in the rows it will never need to move again, then wakes on new data in the field.

A.10 Multi-Branch Absorption

The ship-wake-extend loop (§A.9) lets a single organism continue learning across deployment cycles. A natural next question is whether two organisms trained *independently* on disjoint task streams can be *composed* into a single recipient without retraining either of them. We construct a multi-branch organism that does so, with a shared frozen L0 random projection as the only structural prerequisite.

Architecture. A multi-branch organism comprises a shared frozen L0 (the same Kaiming-initialized random projection used in single-branch trioron, §3) and N parallel *branches*. Each branch is a self-contained donor produced by an independent training run: a frozen L1 substrate, a frozen head slice over its own covered class subset, and the per-class manifold archive $\{(\mu_c, \Sigma_c)\}_{c \in C_b}$ that is already computed for manifold replay (§A.6). Branches must agree on the L0 code space they consume; §A.11 discusses the two protocols we use — a *shared-seed invariant* where all donors are born at one seed, and a *factored handshake* where donors share only the surviving subspace and differ in basis. We enforce non-overlap of class coverage across branches: $C_b \cap C_{b'} = \emptyset$ for $b \neq b'$.

Forward pass. For input x:

1. Project to code space: $z = \sigma(W_{L_0} x + b_{L_0})$ using the shared L0.

2. For each branch b, compute the per-branch head logits $\ell^{(b)} = \text{branch}_b(z)$ over C_b.

3. Compute per-branch generative log-likelihoods from the archive as a uniform mixture of per-class diagonal Gaussians:
$$\log p_b(z) = \log \left(\tfrac{1}{|C_b|} \sum_{c \in C_b} \mathcal{N}(z; \mu_c, \Sigma_c) \right).$$

4. Compute branch gates by softmax over branches:
$$g_b(z) = \frac{\exp(\log p_b(z)/T)}{\sum_{b'} \exp(\log p_{b'}(z)/T)},$$
where T is a temperature ($T = 1$ default). Soft gates (rather than argmax) are deliberate: when archive log-likelihoods on a query are close, multiple branches co-fire and their log-probability contributions are blended.

5. Compose into combined logits over the union $C_\cup = \bigsqcup_b C_b$ via the principled mixture-of-experts log-probability:
$$\text{combined}_c(z) = \log g_{b(c)}(z) + \log \text{softmax}_c(\ell^{(b(c))}),$$
where $b(c)$ is the unique branch covering c.

Why per-branch log-softmax. Each donor's head was trained under its own active-class mask, so absolute logit *magnitudes* across donors are not comparable. Combining raw logits weighted by gates produces a biased argmax at full-softmax. Passing each branch's logits through a log-softmax restricted to its own coverage normalizes the contributions to a common log-probability scale, after which the mixture combine is well-posed. This is an inference-only fix; no parameters are added.

Why the mechanism is zero-shot. Donor weights are byte-valid in the recipient because L0 is shared (matched random projection ⇒ matched code distribution), and donor logits are placed into recipient class-namespace via per-class indexing. The gate is computed analytically from the archive — a generative Expert-Gate-style router (Aljundi et al., 2017) using the manifold buffer trioron already produces. No router training, no fusion-layer learning, no calibration pass; pure paste-and-go.

Optional dream-cycle calibration. For deployments where the full-softmax metric is load-bearing, a tiny additive bias offset can be trained to compensate for residual cross-donor magnitude mismatch. We sample synthetic codes $z \sim \mathcal{N}(\mu_c, \Sigma_c)$ from the union archive, forward through the assembled organism, and apply cross-entropy on the global label. We test two variants:

- *per-branch* (N scalars): one bias broadcast uniformly over each donor's covered slots. Mathematically task-aware-safe (a uniform per-donor shift is monotonic on within-task argmax because active classes within a chained-15 task always belong to a single donor by disjoint coverage).
- *per-class* ($|C_\cup|$ scalars): one bias per union class. Higher capacity, but *can* perturb within-task argmax; trades a small task-aware loss for a larger full-softmax gain.

The dream cycle uses only the donors' own archives — no real-data retraining is performed.

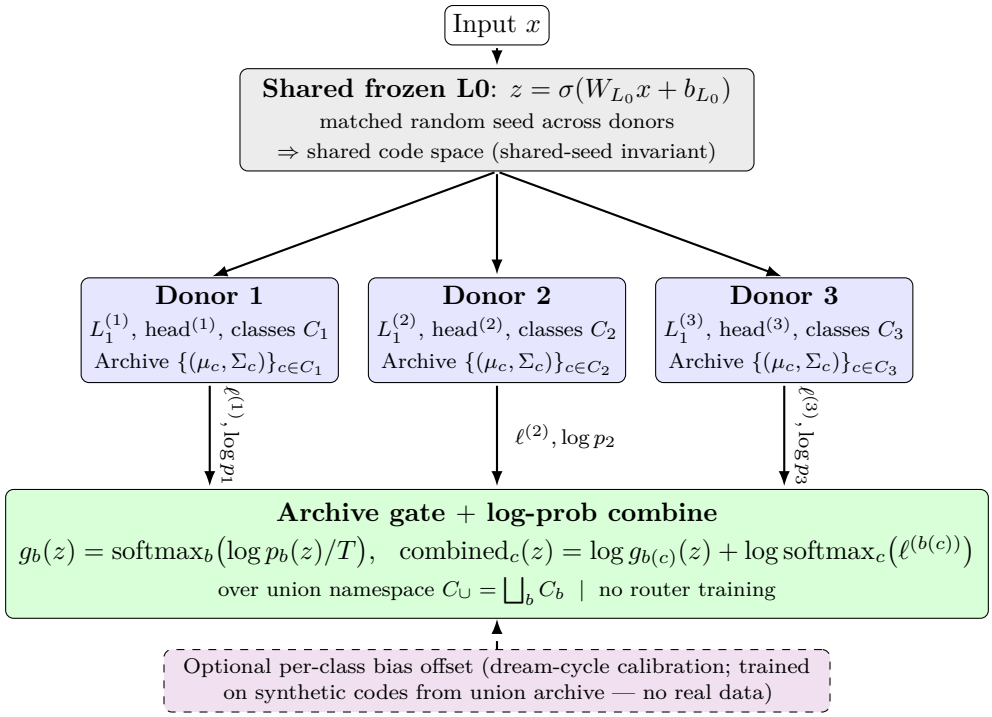

Figure 2: Multi-branch absorption (chimera) at $N = 3$ donors. All donors share the same frozen Gaussian L0 by being born at matching random seeds, so donor weights are byte-valid in the recipient code space (*shared-seed invariant*). Each donor contributes per-class logits $\ell^{(b)}$ and a manifold-archive log-likelihood $\log p_b(z)$; the same archive that serves manifold replay (§A.6) also serves as the analytic Expert-Gate (Aljundi et al., 2017) router. The combine operation places per-branch log-softmax contributions into the union class namespace, weighted by the gate — no router training and no fusion-layer learning. The dashed per-class bias offset is the optional dream-cycle calibration.

A.11 The L0 Handshake

The multi-branch architecture in §A.10 requires that every branch consume the same L0 code space; otherwise donor weights trained against one random projection have no semantic meaning under a different one. We discuss two handshake protocols. The first is the shared-seed invariant assumed above: all donors are born at the same random seed, so $W_{L_0}^{(b)}$ is byte-identical across donors. This is operationally simple but requires coordinated initialization across training runs, which is brittle in any deployment setting where donors are produced independently or by different parties.

The natural alternative is to permit donors to be born at distinct seeds and reconcile their codes at absorb time. The closed-form translator $M = W_B \cdot W_A^+$ that maps donor A's pre-activation codes into donor B's basis is a small linear operator that the recipient can compute from the public projection matrices alone. In practice, however, two independently-drawn 128×784 Gaussian projections produce row-spaces $U_A, U_B \subset \mathbb{R}^{784}$ whose intersection is generically zero-dimensional ($128 + 128 \leq 784$): donor A's L0 captures a different 128-d slice of the input than donor B's. The translator is exact only on U_A, and the residual on a full-rank input is $W_B \cdot (I - P_A) \cdot x$, the component of x that donor A's L0 discarded by construction. We empirically observe that closed-form translation alone recovers 84.8% of the same-seed control task-aware accuracy on a 2-donor chained-15 split (digits + fashion, Table 4); even with a probe-data oracle archive that fixes

the routing-side mismatch, the translator caps near 89.5% because donor B's L1 was trained on full-rank $W_B \cdot x$ and cannot fully recover from the projection bottleneck.

Subspace–basis decomposition. The bottleneck is not an artifact of the translator; it is intrinsic to having two donors disagree on *which* 128-d subspace of input space their L0s preserve. The information loss from $784 \to 128$ is deliberate: it is the information-theoretic floor of the substrate, an irreversible compression that fixes the rank of every L0 representation at 128. The natural question is whether donors can disagree on something that does not also lift that compression bottleneck.

The decomposition that achieves this is structural. Factor the L0 weight as

$$W_{L_0}^{(b)} \;=\; R_b \cdot S, \tag{8}$$

where $S \in \mathbb{R}^{128\times 784}$ is a fixed *protocol-level* subspace selector — a public Gaussian matrix derived once from a hardcoded protocol seed — and $R_b \in \mathbb{R}^{128\times 128}$ is a per-donor orthogonal rotation seeded by the donor's ℓ_b. All donors of the same protocol version project onto the *same* surviving 128-d subspace $\mathrm{row}(S) \subset \mathbb{R}^{784}$; the per-donor randomness lives in the basis chosen within that subspace. Substituting Eq. 8 into the translator construction gives

$$M \;=\; W_B \cdot W_A^{+} \;=\; (R_B S)\cdot(R_A S)^{+} \;=\; R_B \cdot S \cdot S^{+} \cdot R_A^{-1} \;=\; R_B \cdot R_A^{-1}, \tag{9}$$

a pure 128×128 rotation. Because S has full row rank, $S \cdot S^{+} = I_{128}$ and the bottleneck residual cancels identically: for any $x \in \mathbb{R}^{784}$, $M \cdot W_A x = R_B \cdot R_A^{-1} \cdot R_A \cdot S \cdot x = W_B x$ exactly. Donor B's L1 receives its native training distribution after translation, bit for bit. The closed-form pseudoinverse construction is unchanged from the unfactored case — the rotation is recovered automatically by the existing translator code.

The compression floor is preserved: S is the lossy stage and is public by design; the seed of R_b is the only per-donor handshake state, and recipients can regenerate R_b deterministically from a 4-byte integer. Per-donor handshake bandwidth collapses from $\sim 200\,\mathrm{KB}$ ($W_{L_0}^{(b)}$ in FP16) to four bytes; the S matrix is loaded once per recipient and amortized across all donors.

Empirical validation. We retrain a 2-donor split (digits at $\ell = 42$, fashion at $\ell = 43$) under the factored scheme and run the same 5-path bench as the unfactored case (§4.6). Closed-form translation matches the same-seed control to within seed noise and is bit-identical to the publish-W_{L_0} path (Table 4). Routing under the factored translator fires on the correct branch with 95–99% weight on every task ($\Delta \approx +25\,\mathrm{pp}$ from the 60–70% routing-confidence we observed under the unfactored translator), confirming that both the archive-scoring and the L1-forward sides of the absorption pipeline become lossless under Eq. 8.

Cost. Closed-form translation under Eq. 8 costs one 128×784 canonical L0 matmul plus one 128×128 translator matmul per non-canonical donor; at $N = 3$ donors this is $\approx 50\%$ of the compute of N parallel L0 forwards (the publish-W_{L_0} path). Per-donor handshake state is the four-byte ℓ_b seed; the protocol constant S is amortized over all donors of the same protocol version. We treat the protocol seed as a versioned wire-protocol field; bumping it intentionally breaks absorption with previous-version donors (a feature, not a coordination failure).

Path (cross-seed digits + fashion)	Unfactored W	Factored $W = R \cdot S$
A. Same-seed control (upper bound)	0.9821	0.9796
B. Cross-seed naive (no compensation)	0.7408	0.7387
C. Cross-seed random-projection adapter	0.7418	0.7383
D. Cross-seed closed-form translator	0.8325	**0.9800**
E. Cross-seed publish-W_{L_0} (lossless)	0.9819	0.9800
Δ (D vs. same-seed control)	-14.96 pp	**-0.04 pp**

Table 4: Mean task-aware accuracy across the 10 binary tasks of the digits + fashion split, $n = 1$ seed. The same translator code yields a -15 pp gap in the unfactored case and a within-noise gap under the $R \cdot S$ factorization. Path E (the recipient computes each donor's L0 directly from x) is the publish-W_{L_0} reference; its parity with Path D under the factored scheme reflects that the closed-form translator becomes a pure basis rotation (Eq. 9).

Limitations. The factored handshake requires donors to be trained under the factored scheme from initialization onward; existing unfactored donors cannot be retro-translated losslessly because their L1 saw codes drawn from a different surviving subspace. The factorization also changes the L0 weight *distribution* relative to a free Gaussian draw: $R_b \cdot S$ is a random orthogonal rotation of a fixed Gaussian, which is itself Gaussian-distributed with the same covariance, but the cross-donor correlation structure differs from the independent-Gaussian case. Standalone donor accuracy was within seed noise of unfactored donors in our runs (digits 0.977 vs. 0.981, fashion 0.985 vs. 0.984), but a larger evaluation n would be needed to rule out a small training-distribution penalty. The closed-form translator additionally inherits the numerical-floor sensitivity of `torch.linalg.pinv`: we observe $\sim 10^{-12}$ mean-squared error (MSE) in FP32 and $\sim 10^{-7}$ in 16-bit floating-point (FP16), which is adequate for inference but worth measuring on any new deployment-target dtype.

A.12 Experiment Design

Curriculum (chained-15). A 30-class class-incremental benchmark drawn from three datasets in sequence:

- 10 MNIST digit tasks (one new digit class per task);
- 10 Fashion-MNIST tasks (one new garment class per task);
- 10 EMNIST-letters tasks A..J (one new letter class per task).

Each task introduces exactly one new class. At evaluation time the network is asked to classify across all classes seen so far (full-softmax) or restricted to the same task or domain (task-aware, domain-aware). The class-incremental framing forces the network to commit to absolute class identities — there is no task ID at inference — and is the standard hard regime for continual-learning evaluation.

Why class-incremental, not task-incremental. Task-incremental benchmarks supply the task ID at inference, which collapses the problem to learning a per-task classifier. Class-incremental does not, and is therefore the regime where partition-style methods (PackNet, HAT) suffer their characteristic full-softmax collapse. Reporting class-incremental from the start avoids confusing the comparison.

Extension curriculum (chained-23). chained-15 above, plus 8 additional binary EMNIST-letters tasks paired over K..Z (K–L, M–N, ..., Y–Z) introduced at the extension boundary; total 23 class-incremental tasks across 46 classes. Used only in the extension experiment of §4.5.

Network configuration.

- Three layers: input → L0 → L1 → head.
- L0: 784 → 128, frozen-Gaussian random projection (weights drawn from $\mathcal{N}(0, 2/784)$ at init and never updated). See §A.6 for why L0 is frozen.
- L1: 128 → 32 initial, grown adaptively up to a trainable-parameter cap of ≈ 8 K during chained-15 (*cap_bytes* = 32,000 at 4 B/param), lifted to ≈ 16 K at the extension boundary.
- Head: L_1_width → $n_{\text{classes\_so\_far}}$, grown adaptively as new classes appear.
- Activations: ReLU on L1, identity on the head.
- Optimizer: Adam, lr = 3×10^{-3}.
- Epochs: 4 per task. Loss: cross-entropy on the active label set.
- EWC strength: $\beta = 1000$ during waking and dream-replay (matched).

Trioron arms.

- `fixed_ewc_small` — no growth; L1 fixed at the chained-15 cap. Isolates the EWC + manifold-replay contribution at matched parameters.
- `grown_capped_no_dream` — growth under EWC, no dream-time consolidation. Isolates the dream contribution.
- `grown_capped_dream` — growth + dream within cap. The default deployment arm.
- `grown_uncapped_dream` — growth past cap with manifold replay. The upper-bound headline arm.

Competitor families (5). We benchmark against one method from each major continual-learning family:

- *Hippocampal buffer (exemplar rehearsal).* K samples per past class stored. $K \in \{1, 10, 20, 50\}$.
- *PackNet* (Mallya and Lazebnik, 2018). Iterative magnitude pruning with per-task masks. *PackNet matched* matches our trainable-parameter budget; *PackNet standard* uses the original recipe.
- *HAT* (Serrà et al., 2018). Hard binary attention masks per task, gating each layer. *Matched* and *standard* variants as above.
- *Online EWC* (Schwarz et al., 2018). Fisher EMA across tasks with $\gamma = 0.95$; learning-rate and β tuned via grid search on the no-dream loss.
- *LwF + EWC* (Li and Hoiem, 2018; Kirkpatrick et al., 2017). Logit-distillation toward the anchored network on old-class columns, in addition to the EWC penalty.

We are explicit that PackNet and HAT are not designed for class-incremental evaluation without the task-ID supplied at inference; their full-softmax collapse on this benchmark is expected, and the fairer comparison is the task-aware metric, on which they are competitive (§4.2).

Metrics.

- *Full-softmax accuracy* (full): mean across all classes seen so far of top-1 accuracy with the unrestricted softmax. The hard class-incremental metric.
- *Domain-aware accuracy* (domain): top-1 accuracy restricted to classes that share a dataset domain (digits, fashion, letters) with the query.
- *Task-aware accuracy* (task): top-1 accuracy restricted to the classes of the *task* the query belongs to. The right metric for trioron's deployment role of context-gated personalization atop a frozen LLM.
- *Forgetting* (forget): mean across past tasks of (accuracy at the time the task was first introduced) − (final-task accuracy on that task); positive values indicate forgetting, negative values indicate that past-class accuracy improved over the remainder of the curriculum.
- *σ-vs-baseline*: paired Cohen's $d_z = \text{mean}(\Delta_{\text{seed}})/\,\text{std}(\Delta_{\text{seed}})$ across the random-seed cohort (10 seeds for headline panels; 3 seeds for the dream-archive panel), reported alongside absolute Δ. We are explicit that σ values can be inflated by tight per-arm seed-to-seed std rather than by a large absolute effect; we therefore report absolute Δ as the primary number and σ as a secondary tightness indicator.

Storage accounting. We report deployment storage in two columns: BF16 baseline (the honest device-side number, since BF16 is the inference dtype) and FP32 baseline (the training-time number). The manifold buffer is counted at FP32 (per-class μ and σ, $2 \cdot L_0 \cdot 4 = 1024$ bytes/class). All headline storage numbers in §4 use the BF16 baseline.

Reproducibility. All benches run on commodity CPU. Each seed is fully deterministic given the seed value, modulo a known torch nondeterminism in archive-trigger timing (see §4.4 for the v1/v2 reconciliation). Launcher scripts under `experiments/` accept explicit seed lists; bench logs and CSVs from every reported run are committed to the public repository.

B Supporting Data

This appendix collects the per-arm and per-competitor data tables that the main-text narrative summarizes and points at. They are reported here rather than inline so the Results section reads as a narrative argument rather than a table-by-table commentary.

B.1 Per-seed paired comparisons within trioron arms

Forward-pointer from §4.1: this is the internal-ablation evidence that motivates `grown_uncapped_dream` as the headline arm.

Table 5: Paired comparisons against `grown_uncapped_dream`, $n = 10$ paired by seed. Absolute Δ is the primary number; σ is reported as a secondary tightness indicator.

comparison	metric	mean Δ	σ
`fixed_ewc_small` vs `grown_uncapped_dream`	full	-0.023	-1.79
`fixed_ewc_small` vs `grown_uncapped_dream`	domain	-0.024	-2.55
`fixed_ewc_small` vs `grown_uncapped_dream`	task	-0.012	-4.63
`grown_capped_no_dream` vs `grown_uncapped_dream`	full	-0.017	-1.88
`grown_capped_no_dream` vs `grown_uncapped_dream`	domain	-0.017	-1.40
`grown_capped_no_dream` vs `grown_uncapped_dream`	task	-0.010	-4.73
`grown_capped_dream` vs `grown_uncapped_dream`	full	-0.017	-1.05
`grown_capped_dream` vs `grown_uncapped_dream`	domain	-0.020	-1.25
`grown_capped_dream` vs `grown_uncapped_dream`	task	-0.004	-0.87

Table 6: Paired absolute Δ of manifold-grown vs each competitor (positive = ours wins), $n = 10$ paired by seed. Absolute Δ is the primary number; σ in parentheses is the secondary tightness indicator.

comparison	full Δ (σ)	domain Δ (σ)	task Δ (σ)
vs PackNet matched	$+0.522$ ($+20.3\sigma$)	$+0.477$ ($+24.5\sigma$)	$+0.031$ ($+1.9\sigma$)
vs PackNet standard	$+0.555$ ($+24.5\sigma$)	$+0.535$ ($+30.5\sigma$)	$+0.090$ ($+2.4\sigma$)
vs HAT matched	$+0.481$ ($+28.2\sigma$)	$+0.462$ ($+15.7\sigma$)	$+0.025$ ($+5.1\sigma$)
vs HAT standard	$+0.530$ ($+25.6\sigma$)	$+0.515$ ($+17.4\sigma$)	$+0.018$ ($+0.8\sigma$)
vs Online EWC	$+0.416$ ($+14.4\sigma$)	$+0.322$ ($+17.5\sigma$)	$+0.006$ ($+1.5\sigma$)
vs LwF + EWC	$+0.410$ ($+10.1\sigma$)	$+0.319$ ($+8.2\sigma$)	-0.003 (-1.3σ)

B.2 Per-competitor paired absolute Δ vs manifold-grown

Forward-pointer from §4.2: per-seed paired view of the five-family competitor sweep. Replaces the unpaired family means with cross-seed-corrected effect sizes.

B.3 Dream archive accuracy and deployment storage

Forward-pointer from §4.4: per-arm evidence that Phase 2 mixed-precision is lossless within $\Delta \leq 0.0008$, and that the BF16 net collapses to $\sim$130 KB after archive + int8.

Table 7: chained-15 with archive Phase 1 + Phase 2, $n = 3$ (dream-archive panel not yet rerun at $n = 10$).

arm	full v2	full Δ vs no-archive	task Δ
`fixed_ewc_small`	0.572 ± 0.004	-0.013	$+0.001$
`grown_capped_no_dream`	0.559 ± 0.004	-0.019	-0.003
`grown_capped_dream`	0.572 ± 0.009	-0.015	$+0.006$
`grown_uncapped_dream`	0.617 ± 0.004	$+0.013$	$+0.006$

Table 8: Deployment storage, BF16 baseline, $n = 3$ means (dream-archive panel not yet rerun at $n = 10$).

arm	BF16 baseline	BF16 + int8 archived	Δ
`fixed_ewc_small`	213.7 KB	130.4 KB	−39%
`grown_capped_no_dream`	213.7 KB	128.9 KB	−40%
`grown_capped_dream`	211.2 KB	127.5 KB	−40% ★
`grown_uncapped_dream`	224.9 KB	137.2 KB	−39%

B.4 Architecture progression across the ship-wake-extend loop

Forward-pointer from §4.5: structural evidence that the loop preserves archived rows from the previous deployment cycle while allowing the plastic envelope to expand against the new task budget.

Table 9: Architecture progression across the ship-wake-extend loop (1 seed).

phase	layers	trainable	archived	plastic
start	$(128, 32, 2)$	4 194	[0, 0, 0]	4 194
chained-15 done	$(128, 48, 30)$	7 662	[104, 32, 0]	3 534
chained-23 done	$(128, 80, 46)$	14 046	[128, 40, 0]	8 886

References

Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. Expert gate: Lifelong learning with a network of experts. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7120–7129, 2017.

Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.

C. David Allis and Thomas Jenuwein. The molecular hallmarks of epigenetic control. *Nature Reviews Genetics*, 17(8):487–500, 2016.

Andrew J. Bannister and Tony Kouzarides. Regulation of chromatin by histone modifications. *Cell Research*, 21(3):381–395, 2011.

David P. Bartel. MicroRNAs: Target recognition and regulatory functions. *Cell*, 136(2):215–233, 2009.

Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2net: Accelerating learning via knowledge transfer. In *International Conference on Learning Representations (ICLR)*, 2016.

Scott E. Fahlman and Christian Lebiere. The cascade-correlation learning architecture. In *Advances in Neural Information Processing Systems (NIPS)*, volume 2, 1990.

Marcus Frean. The upstart algorithm: A method for constructing and training feedforward neural networks. *Neural Computation*, 2(2):198–209, 1990.

Brett J. Kagan, Andy C. Kitchen, Nhi T. Tran, Forough Habibollahi, Moein Khajehnejad, Bradyn J. Parker, Anjali Bhat, Ben Rollo, Adeel Razi, and Karl J. Friston. In vitro neurons learn and exhibit sentience when embodied in a simulated game-world. *Neuron*, 110(23):3952–3969.e8, 2022. doi: 10.1016/j.neuron.2022.09.001.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114.

Teuvo Kohonen. Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1):59–69, 1982.

Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tim Althoff, Noah A. Smith, and Luke Zettlemoyer. Branch-train-merge: Embarrassingly parallel training of expert language models, 2022.

Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2018. doi: 10.1109/TPAMI.2017.2773081.

David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7765–7773, 2018.

Nature Editorial. Tools such as ChatGPT threaten transparent science; here are our ground rules for their use. *Nature*, 613:612, 2023. doi: 10.1038/d41586-023-00191-1.

Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2016.

Gerhard Schratt. microRNAs at the synapse. *Nature Reviews Neuroscience*, 10(12):842–849, 2009.

Jonathan Schwarz, Jelena Luketina, Wojciech M. Czarnecki, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & Compress: A scalable framework for continual learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*. PMLR, 2018.

Joan Serrà, Dídac Surís, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 4548–4557. PMLR, 2018.

Zachary D. Smith and Alexander Meissner. DNA methylation: roles in mammalian development. *Nature Reviews Genetics*, 14(3):204–220, 2013.

Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Daniel Li, Wen-tau Yih, Jason Weston, and Xian Li. Branch-train-MiX: Mixing expert LLMs into a mixture-of-experts LLM, 2024.

Giulio Tononi and Chiara Cirelli. Sleep and the price of plasticity. *Neuron*, 81(1):12–34, 2014.

Lewis Wolpert. Positional information and the spatial pattern of cellular differentiation. *Journal of Theoretical Biology*, 25(1):1–47, 1969.

Hongxu Yin, Pavlo Molchanov, Jose M. Alvarez, Zhizhong Li, Arun Mallya, Derek Hoiem, Niraj K. Jha, and Jan Kautz. Dreaming to distill: Data-free knowledge transfer via DeepInversion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

Chris Zielinski, Margaret A. Winker, Rakesh Aggarwal, Lorraine E. Ferris, Markus Heinemann, Jose Florencio Lapena, Sanjay A. Pai, Edsel Ing, Leslie Citrome, Murad Alam, Maurice Voight, and Farrokh Habibzadeh. Chatbots, generative AI, and scholarly manuscripts: WAME recommendations. *World Association of Medical Editors*, 2023.